

UD 01 - Programación avanzada en Java y acceso a datos mediante ficheros

Clases genéricas

Se trata de una clase parametrizada sobre uno o más tipos. El tipo se asigna en tiempo de compilación

```
1 public class Box {
2     private Object object;
3
4     public void set(Object object) {
5         this.object = object;
6     }
7     public Object get() {
8         return object;
9     }
10 }
```

```
1 public class Box<T> {
2     private T object;
3
4     public void set(T object) {
5         this.object = object;
6     }
7
8     public T get() {
9         return object;
10    }
11 }
```

El código de la izquierda es más propenso a producir errores.

Otro ejemplo:

```
1 public class Par<T, S> {
2     private T obj1;
3     private S obj2;
4
5     // Resto de la clase
6
7 }
```

Los nombres de tipos de parámetros más usados son:

- E (element, elemento)
- K (key, clave)
- N (number, número)
- T (type, tipo)
- V (value, valor)
- S, U, V, ... (2º, 3º, 4º, ... tipo)

Para instanciar un objeto genérico, tenemos que indicar los tipos dos veces.

```
Par<String, String> pareja2 = new Par<String, String>("Hola", "Mundo");
```

Este estilo es muy verboso. Desde Java SE 7 tenemos el operador <>

```
Par<String, String> pareja2 = new Par<>("Hola", "Mundo");
```

Podemos indicar que el tipo parametrizado sea uno en particular (o sus derivados).

```
public class NumericBox<T extends Number>
public class StrangeBox <T extends A>
```

¿Qué es un tipo comodín <?> en Java?

Los tipos comodín (wildcard), aparecen con el concepto de programación genérica. En castellano también los puedes encontrar como tipo salvaje.

Imagina que tienes una caja. Esa caja puede contener diferentes tipos de objetos: manzanas, naranjas, libros, etc. Sin embargo, no sabemos exactamente qué tipo de objeto hay dentro hasta que abrimos la caja.

En Java, los tipos comodín son como esa caja. Representan un tipo desconocido, pero que cumple con ciertas restricciones. Se utilizan cuando no sabemos el tipo exacto de un objeto en tiempo de compilación, pero necesitamos realizar algunas operaciones con él.

¿Para qué sirven los tipos comodín?

Mayor flexibilidad: Permiten escribir código más genérico y reutilizable, ya que no están limitados a un tipo de dato específico.

Mayor seguridad: Ayudan a evitar errores de tiempo de ejecución relacionados con tipos incompatibles.

Colecciones: Son especialmente útiles al trabajar con colecciones como List, Set y Map, donde los elementos pueden ser de diferentes tipos.

```
List<?> myList; // Una lista que puede contener cualquier tipo de objeto
```

En este ejemplo, myList puede contener una lista de números enteros, una lista de cadenas o incluso una lista de objetos de una clase personalizada. Sin embargo, no podemos agregar elementos directamente a myList porque no sabemos su tipo exacto.

Tipos de tipos comodín:

<?>: Tipo comodín sin restricciones. Puede representar cualquier tipo.

<? extends T>: Tipo comodín con límite superior. Solo puede representar tipos que sean subtipos de T.

<? super T>: Tipo comodín con límite inferior. Solo puede representar tipos que sean supertipos de T.

```
List<? extends Number> numbersList; // Solo puede contener números
List<? super Integer> integerList; // Puede contener Integer y sus
supertipos (como Number o Object)
```

Diferencias entre tipos genéricos y comodín o salvajes:

Imagina que tenemos una clase Caja que puede contener cualquier tipo de objeto:

```
class Caja<T> {  
    private T contenido;  
  
    public void setContenido(T contenido) {  
        this.contenido = contenido;  
    }  
  
    public T getContenido() {  
        return contenido;  
    }  
}
```

Aquí, T es un tipo genérico. Esto significa que podemos crear cajas que contengan diferentes tipos de objetos:

```
Caja<String> cajaDePalabras = new Caja<>();  
cajaDePalabras.setContenido("Hola");  
  
Caja<Integer> cajaDeNumeros = new Caja<>();  
cajaDeNumeros.setContenido(42);
```

En este caso, sabemos exactamente el tipo de objeto que contiene cada caja (String o Integer).

Ahora, veamos un ejemplo con un tipo salvaje:

```
List<?> listaMisteriosa;
```

Aquí, listaMisteriosa es una lista que puede contener cualquier tipo de objeto, pero no sabemos exactamente cuál es ese tipo. Podría ser una lista de String, Integer, o cualquier otra clase.

¿Cuál es la diferencia?

Tipo genérico: Define un tipo específico que se utilizará en una clase o método. Nos permite crear objetos con tipos concretos.

Tipo salvaje: Representa un tipo desconocido en tiempo de compilación. Nos proporciona flexibilidad, pero limita las operaciones que podemos realizar con esos objetos.

```
public static void imprimirElementos(List<?> lista) {  
    for (Object elemento : lista) {  
        System.out.println(elemento);  
    }  
}
```

Este método puede imprimir los elementos de cualquier lista, sin importar el tipo de los elementos.

Conclusión:

Los tipos genéricos nos permiten crear clases y métodos que trabajan con tipos específicos.

Los tipos salvajes nos permiten crear código más genérico, pero con menos funcionalidades.

Comparable y Comparator

Comparable y Comparator son dos interfaces en Java utilizadas para comparar objetos. Aquí están las principales diferencias:

Comparable

- La interfaz Comparable se encuentra en el paquete java.lang y tiene un único método llamado compareTo().
- Si quieres que los objetos de una clase sean comparables por defecto, entonces esa clase debe implementar Comparable.
- El método compareTo() compara el objeto actual con el objeto especificado y devuelve un entero negativo, cero o un entero positivo si el objeto actual es menor que, igual a, o mayor que el objeto especificado.
- Comparable es útil cuando tienes una única fuente de ordenamiento.

Comparator

- La interfaz Comparator se encuentra en el paquete java.util y tiene dos métodos: compare() y equals().
- Puedes crear una clase separada que implemente "Comparator", y luego usar un objeto de esa clase para comparar objetos de otra clase, lo que significa que no necesitas modificar la clase que quieres comparar.
- El método compare() compara sus dos argumentos y devuelve un entero negativo, cero o un entero positivo si el primer argumento es menor que, igual a, o mayor que el segundo.
- Comparator es útil cuando tienes múltiples fuentes de ordenamiento. Por ejemplo, puedes tener un Comparator que ordene por nombre y otro que ordene por edad.

En resumen, Comparable es para una ordenación "natural" y por defecto de los objetos de una clase, mientras que Comparator es para ordenaciones personalizadas.

Otra explicación:

Muchas operaciones entre objetos nos obligan a compararlos: buscar, ordenar, ... Los tipos primitivos y algunas clases ya implementan su orden (natural, lexicográfico). Para nuestras clases (modelo) tenemos que especificar el orden con el que las vamos a tratar

Comparable:

Se trata de un interfaz sencillo. Recibe un objeto del mismo tipo. Devuelve 0 si son iguales, un valor negativo si es menor, y uno positivo si es mayor. Nos sirve para indicar el orden principal de una clase.

```
public interface Comparable<T> {  
    // 0 si es igual, 1 si es mayor, -1 si es menor.  
    public int compareTo(T o);  
}
```

```
}
```

Comparator:

Se trata de un interfaz sencillo. Recibe dos argumentos. Devuelve 0 si son iguales, un valor negativo si es menor, y uno positivo si es mayor. Nos sirve para indicar un orden puntual, diferente al orden principal de una clase.

```
public interface Comparator<T> {  
    public int compare(T o1, T o2);  
}
```

Ejemplos:

Diferencia entre Comparable y Comparator en Java.

Comparable

```
import java.util.*;  
  
class Student implements Comparable<Student> {  
    int id;  
    String name;  
  
    Student(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
  
    public int compareTo(Student s) {  
        return this.id - s.id;  
    }  
  
    public String toString() {  
        return this.id + " " + this.name;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        List<Student> list = new ArrayList<>();  
        list.add(new Student(3, "Alice"));  
        list.add(new Student(1, "Bob"));  
        list.add(new Student(2, "Charlie"));  
  
        Collections.sort(list);  
  
        System.out.println(list);  
    }  
}
```

En este ejemplo, la clase Student implementa Comparable<Student>. Cuando llamamos a Collections.sort(list), los objetos Student en la lista se ordenan por su id porque eso es lo que especificamos en el método compareTo().

Comparator

```
import java.util.*;

class Student {
    int id;
    String name;

    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public String toString() {
        return this.id + " " + this.name;
    }
}

class StudentNameComparator implements Comparator<Student> {
    public int compare(Student s1, Student s2) {
        return s1.name.compareTo(s2.name);
    }
}

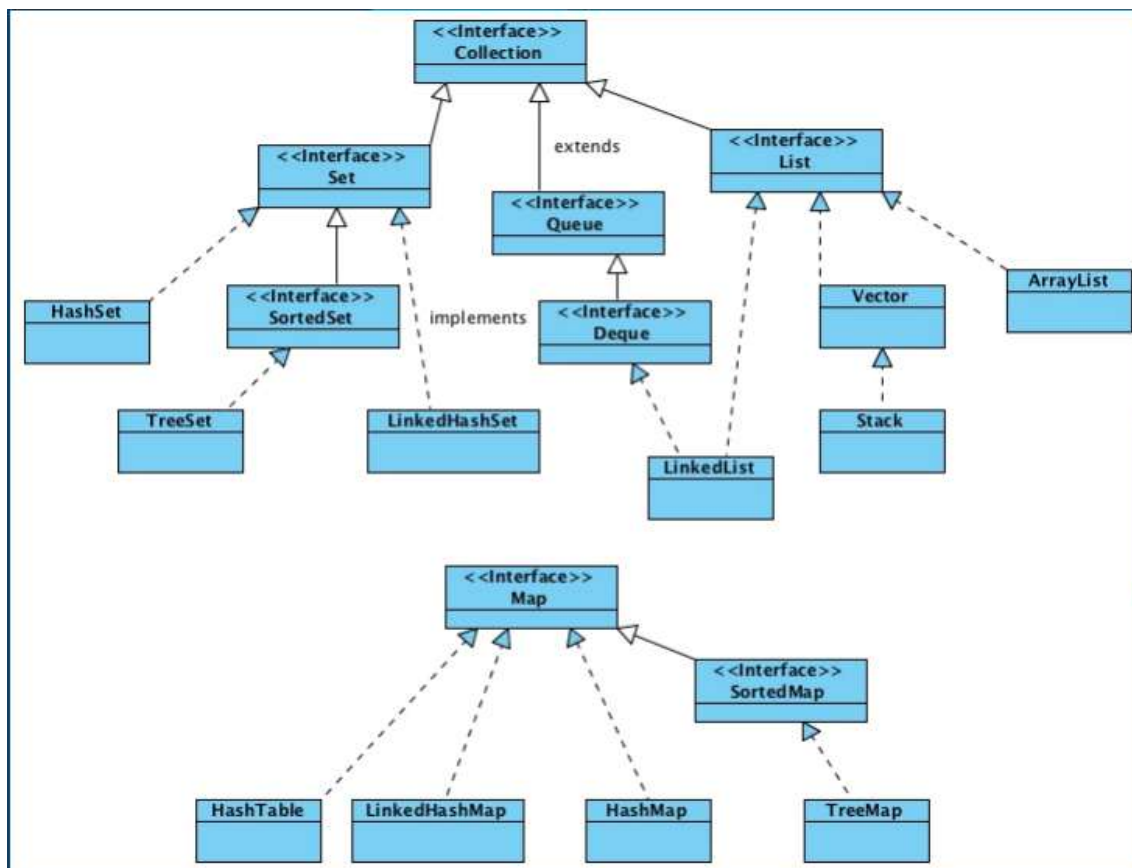
public class Main {
    public static void main(String[] args) {
        List<Student> list = new ArrayList<>();
        list.add(new Student(3, "Alice"));
        list.add(new Student(1, "Bob"));
        list.add(new Student(2, "Charlie"));

        Collections.sort(list, new StudentNameComparator());

        System.out.println(list);
    }
}
```

En este ejemplo, la clase `Student` no implementa `Comparable<Student>`. En su lugar, creamos una clase separada `StudentNameComparator` que implementa `Comparator<Student>`. Cuando llamamos a `Collections.sort(list, new StudentNameComparator())`, los objetos `Student` en la lista se ordenan por su `name` porque eso es lo que especificamos en el método `compare()`.

Colecciones



Ejercicios Básicos:

1 List

- Ej1.** Crear una lista de Strings y añadir elementos a ella. Luego, imprimir todos los elementos de la lista usando un bucle for-each.
- Ej2.** Dada una lista de números enteros, escribir una función que devuelva una nueva lista que contenga solo los números pares de la lista original.

Ej3. Dada una lista de Strings, escribir una función que devuelva la longitud del string más largo en la lista.

2 Set

Ej1. Crear un Set de Strings y añadir elementos a él. Luego, imprimir todos los elementos del Set. ¿Qué observas acerca del orden de los elementos?

Ej2. Dada una lista de números enteros, escribir una función que devuelva un Set que contenga solo los números únicos de la lista original.

Ej3. Dada una lista de Strings, escribir una función que devuelva un Set que contenga solo los Strings únicos de la lista original.

3 Map

Ej1. Crear un Map que asocie nombres de países con sus capitales. Luego, imprimir todos los pares de clave-valor del Map.

Ej2. Dada una lista de Strings, escribir una función que devuelva un Map donde las claves son los Strings únicos de la lista y los valores son el número de veces que cada String aparece en la lista.

Ej3. Dada una lista de estudiantes (donde cada estudiante es un objeto con propiedades como nombre, edad, grado, etc.), escribir una función que devuelva un Map donde las claves son los nombres de los estudiantes y los valores son los objetos de los estudiantes.